

A SMALL COMPANION FOR THE I LESSONS

I Learn

A gentle walk from C's machine-shaped ideas into I's
declarations, generics, and reflection.

Open beside the lessons in `src/lessons` . Print it when you want paper margins.

C-SHAPED, I-FLAVORED

CONTENTS

The Path

This booklet is a map, not the full manual. The real book is the set of tiny programs you can compile and inspect.

- | | |
|----------------------------------|--------------------------|
| 1. The Core Idea | what C gives us |
| 2. How To Read A Lesson | compile, inspect, repeat |
| 3. The Lesson Trail | fourteen little programs |
| 4. Where I Starts To Be I | generics and reflection |
| 5. The Check Ritual | one command |

The examples use the packaged compiler from `C:\devel\i-windows-x64`, the same way a separate user project should.

The Core Idea

C is not C because of punctuation. C is C because it keeps the physical program visible: memory, addresses, contiguous arrays, structs as layout, enums as named integers, and procedures as callable symbols.

I keeps that model. It does not try to hide the machine. It tries to make the shape of the program easier to write, easier to check, and easier for tools to understand.

Memory

Values live in storage. Pointers name storage. Arrays put values next to each other.

Layout

A struct is a promise about fields and offsets. That promise matters when talking to C.

Compilation

I lowers to C. The generated C is useful evidence, not a secret.

```
Point: struct = {  
    x: f32;  
    y: f32;  
}
```

How To Read A Lesson

Each lesson is a full program. Read the comments, compile it, and then look at the generated C when the lowering is interesting.

The rule of thumb is small and reliable:

```
read the I
compile the I
inspect the C
change one thing
compile again
```

That loop teaches more than a big reference page. It makes every feature answer a practical question.

The lesson files are intentionally short. When a file grows too much, it stops being a lesson and becomes a project.

The Lesson Trail

00

Hello World

The smallest complete I program.

01

Values And Expressions

Numbers, casts, operators, and output.

02

Control Flow

Loops, conditionals, switch cases, and scoped case blocks.

03

Arrays And Pointers

Contiguous storage, addresses, and mutation through a pointer.

04

Structs And Enums

Named layouts and named integer states.

05

Procedures And Callbacks

Proc pointers in arrays and structs.

06

The C Surface

`cinclude` , `external` , and checked calls into C.

07

Standard Print

Imports from `std` beside the installed compiler.

08

Arenas And Array

Explicit allocation and a generic standard container.

09

Generic Box

A tiny generic struct and owner-qualified generic procedures.

10

Enum-Sized Tables

Tables sized by `Enum<>.value_count`.

11

Reflection Metadata

Names, sizes, field counts, and pointer depth.

12

Reflection Fields

Walking reflected fields and reading attributes.

13

Metaprogramming Shape

Generic instantiations and their reflected concrete names.

Where I Starts To Be I

The first lessons are mostly C ideas in I syntax. The later lessons show why the syntax is worth having.

Generics let one source definition become concrete code:

```
Box: struct<T> = {  
    value: T;  
}  
  
Box<T>make: proc<T>(value: T)->Box<T> = {  
    box: Box<T> = {};  
    box.value = value;  
    return box;  
}
```

Reflection gives generated metadata for the data you already wrote:

```
printf("%s %llu\n", Payload<>.name, Payload<>.field_count);
```

The spirit is concrete metaprogramming: less handwritten boilerplate, more checkable generated shape.

CHAPTER 5

The Check Ritual

Use the check script whenever you change a lesson:

```
powershell -ExecutionPolicy Bypass -File scripts/check_all.ps1
```

It finds `I.exe` through `I_HOME`, then `PATH`, then `C:\devel\i-windows-x64\I.exe`.
It compiles every lesson into `build/gen`.

If all is well, it ends with:

```
ok 14 lessons
```

A learning repo should keep this ritual boring. The interesting part is the source, not the build ceremony.

Related notes: [how to read](#), [what C teaches](#), and [generics and reflection](#).

CHECK RITUAL